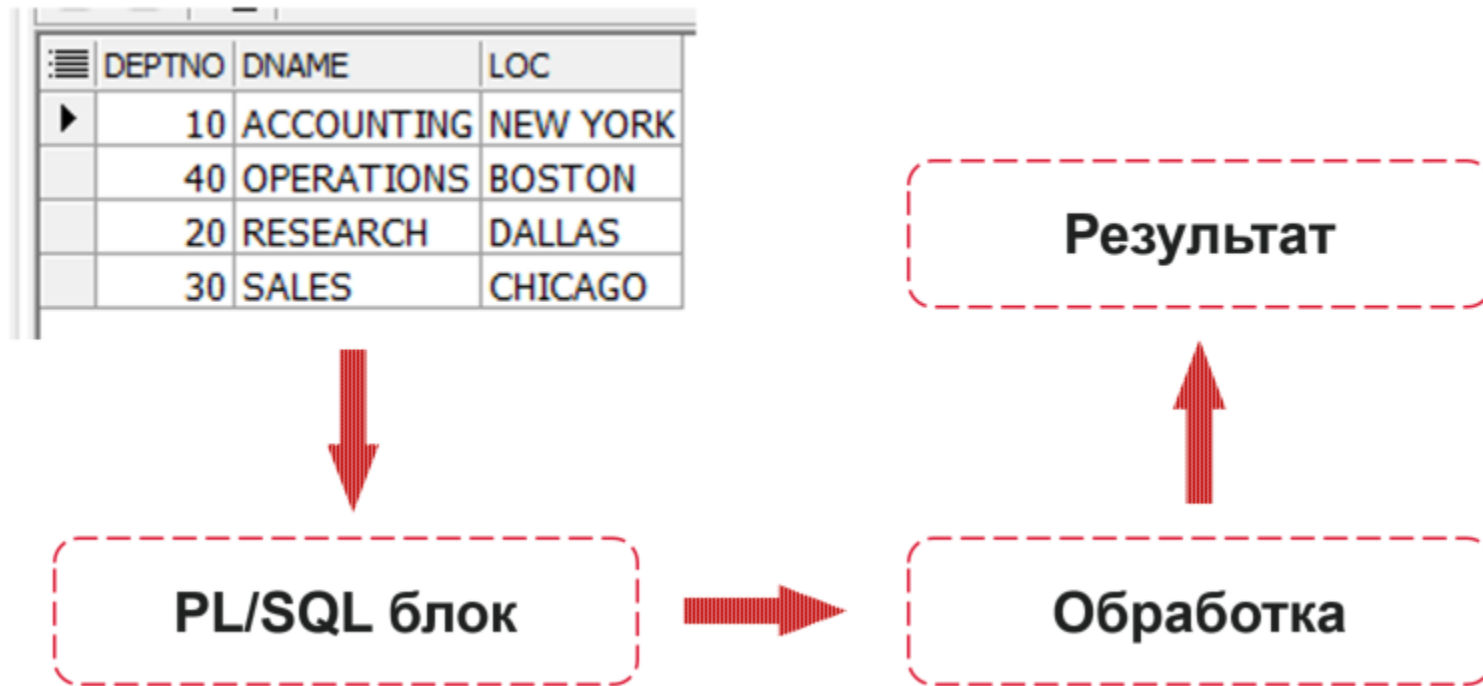




КУРСОРЫ

Курсоры

Курсор используется для получения строк/строки, возвращаемых запросом



Курсоры

Курсор

ЯВНЫЙ

Требует объявления

Неявный

Не требует объявления

Неявный курсор

```
SELECT ENAME INTO VENAME  
FROM EMP  
WHERE EMPNO = 7839;
```

NOTICE: KING

Неявный курсор

```
DO $$  
Declare  
    vEname varchar(100) ;  
Begin  
    select Ename into vEname from emp where Empno = 7839;  
    RAISE NOTICE '%', vEname;  
end;  
$$
```

NOTICE: KING

Неявный курсор

%TYPE – ссылочный тип

DECLARE

VENAME EMP.ENAME%TYPE;

BEGIN

Неявный курсор

- **%TYPE** – ССЫЛОЧНЫЙ ТИП

```
DO $$  
Declare  
    vEname EMP.ENAME%TYPE;  
Begin  
    select Ename into vEname from emp where Empno = 7839;  
    RAISE NOTICE  
        '%', vEname;  
end;  
$$
```

NOTICE: KING

Неявный курсор

```
select Ename, JOB, HIREDATE  
into vEname, vJOB, vHIREDATE  
from emp  
where Empno = 7839;
```


Неявный курсор

```
DO $$
```

```
Declare
```

```
  vEname varchar(100);
```

```
  vJOB    varchar(100);
```

```
  vHIREDATE date;
```

```
Begin
```

```
  select Ename, JOB, HIREDATE into vEname, vJOB, vHIREDATE
```

```
    from emp
```

```
  where Empno = 7839;
```

```
  RAISE NOTICE 'vEname = %, vJOB = %, vHIREDATE = %',
```

```
    vEname, vJOB, vHIREDATE;
```

```
end;
```

```
$$
```

```
NOTICE:  vEname = KING, vJOB = PRESIDENT, vHIREDATE = 1981-11-17
```

Неявный курсор

```
DO $$  
Declare  
    vEname EMP.ENAME%TYPE;  
    vJOB EMP.JOB%TYPE;  
    vHIREDATE EMP.HIREDATE%TYPE;  
Begin  
    select Ename, JOB, HIREDATE into vEname, vJOB, vHIREDATE  
        from emp  
        where Empno = 7839;  
    RAISE NOTICE 'vEname = %, vJOB = %, vHIREDATE = %',  
        vEname, vJOB, vHIREDATE;  
end;  
$$
```

NOTICE: KING

Неявный курсор

%ROWTYPE – курсорная
переменная

Declare

vEMP

emp%ROWTYPE;

Begin

Неявный курсор

```
select * into vEMP from emp  
where Empno = 7839;
```

Неявный курсор

- **%ROWTYPE** – курсорная переменная

```
DO $$  
Declare  
    vEMP      emp%ROWTYPE;  
Begin  
    select * into vEMP from emp where Empno = 7839;  
    RAISE NOTICE 'Ename = %, JOB = %', vEMP.Ename, vEMP.JOB;  
end;  
$$
```

NOTICE: Ename = KING, JOB = PRESIDENT

Неявный курсор

- **RECORD** – запись

```
Declare  
vRowEMP record;  
Begin
```

Неявный курсор

```
select Ename, JOB, HIREDATE  
HIREDATE into vRowEMP  
from emp  
where Empno = 7839;
```

Неявный курсор

```
RAISE NOTICE 'vEname = %,
%, vJOB = %, vHIREDATE = %,
%';
```

```
vRowEMP.Ename,  
vRowEMP.JOB,  
vRowEMP.HIREDATE;
```


Неявный курсор

```
DO $$  
Declare  
    vRowEMP record;  
Begin  
    select Ename, JOB, HIREDATE into vRowEMP  
    from emp  
    where Empno = 7839;  
    RAISE NOTICE 'vEname = %, vJOB = %, vHIREDATE = %',  
        vRowEMP.Ename, vRowEMP.JOB, vRowEMP.HIREDATE;  
end;  
$$
```

- **RECORD** - запись

NOTICE: Ename = KING, JOB = PRESIDENT

Неявный курсор

```
select avg(SAL) into vSal  
from emp  
where DEPTNO = 30;
```

NOTICE: vSal = 1566.666666666666666666666667

Неявный курсор

```
select SAL into vSal  
from emp  
where DEPTNO = 30;
```

?

Неявный курсор

NOTICE: vSal = 1600.00

```
select SAL from emp  
where DEPTNO = 30;
```

	sal numeric (7,2) 
1	<u>1600.00</u>
2	1250.00
3	1250.00

Неявный курсор

```
select SAL into vSal  
from emp  
where DEPTNO = 50;
```

?

Неявный курсор

NOTICE: vSal = <NULL>

Неявный курсор

```
select SAL into STRICT vSal  
from emp
```

- where DEPTNO = 30;
- where DEPTNO = 50;

Неявный курсор

```
FOR i in (select * from emp) LOOP  
  RAISE NOTICE  
    'Ename = %, JOB = %', i.Ename;  
END LOOP;
```


Неявный курсор

```
DO $$  
declare  
    i record;  
begin  
    FOR i in (select * from emp) LOOP  
        RAISE NOTICE  
            'Ename = %, JOB = %', i.Ename, i.JOB;  
    END LOOP;  
end;  
$$
```

Явный курсор

Объявление переменных

Объявление курсора

Открытие курсора

Выбор строк

Использование полученных значений

Закрытие курсора

Явный курсор

declare

cEMP CURSOR FOR

SELECT Empno, Ename

FROM emp;

i record;

Явный курсор

FOR i in cEMP LOOP

RAISE NOTICE

'Ename = %, JOB = %', i.Ename;

END LOOP;

Явный курсор

```
DO $$
declare
    cEMP CURSOR FOR
        SELECT Empno, Ename, Job
        FROM emp;
    i record;
begin
    FOR i in cEMP LOOP
        RAISE NOTICE ' Empno = %, Ename = %, Job = %',
            i.Empno, i.Ename, i.Job;
    END LOOP;
end;
$$
```

NOTICE: Empno = 7369, Ename = SMITH, Job = CLERK
NOTICE: Empno = 7499, Ename = ALLEN, Job = SALESMAN
NOTICE: Empno = 7521, Ename = WARD, Job = SALESMAN

Явный курсор

```
declare  
cEMP CURSOR FOR  
  SELECT Empno, Ename  
  FROM emp  
 where SAL >= 3000;
```

Явный курсор

```
declare  
cEMP CURSOR (pSAL int) FOR  
  SELECT Empno, Ename  
  FROM emp  
  where SAL >= pSAL;
```

Явный курсор

FOR i in cEMP(3000) LOOP

...

FOR i in cEMP(5000) LOOP

...

Явный курсор

```
DO $$
declare
    cEMP CURSOR (pSal int) FOR
        SELECT Empno, Ename, Job, Sal
        FROM emp
        where sal >= pSal;
    i record;
begin
    FOR i in cEMP(3000) LOOP
        RAISE NOTICE '1 part Empno = %, Ename = %, Job = %, Sal = %',
            i.Empno, i.Ename, i.Job, i.Sal;
    END LOOP;
    FOR i in cEMP(5000) LOOP
        RAISE NOTICE '2 part Empno = %, Ename = %, Job = %, Sal = %',
            i.Empno, i.Ename, i.Job, i.Sal;
    END LOOP;
end;
$$
```

Явный курсор

Data Output Messages Notifications

```
NOTICE:  1 part Empno = 7788, Ename = SCOTT, Job = ANALYST, Sal = 3000.00
NOTICE:  1 part Empno = 7839, Ename = KING, Job = PRESIDENT, Sal = 5000.00
NOTICE:  1 part Empno = 7902, Ename = FORD, Job = ANALYST, Sal = 3000.00
NOTICE:  2 part Empno = 7839, Ename = KING, Job = PRESIDENT, Sal = 5000.00
```

Явный курсор

Выбор строк

FETCH имя_курсора INTO переменная[, переменная ...])

где

- имя_курсора определяет имя курсора;
- переменная является переменной, в которую записываются значения из оператора SELECT курсора.

**FETCH cEMP INTO
vEmpNo, vEname, vJob;**

Явный курсор

OPEN cEMP;

LOOP

FETCH cEMP

INTO vEmpNo, vEname, vJob;

EXIT WHEN NOT FOUND;

RAISE NOTICE '1 part Empno = %, Ename = %, Job = %', vEmpno, vEname, vJob;

END LOOP;

CLOSE cEMP;

Явный курсор

```
DO $$
declare
    vEmpNo    EMP.EMPNO%TYPE;
    vEname    EMP.ENAME%TYPE;
    vJob      EMP.JOB%TYPE;
    cEMP CURSOR FOR
        SELECT Empno, Ename, Job FROM emp where sal >= 3000;
begin
    OPEN cEMP;
    LOOP
        FETCH cEMP
        INTO vEmpNo, vEname, vJob;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE '1 part Empno = %, Ename = %, Job = %',
            vEmpno, vEname, vJob;
    END LOOP;
    CLOSE cEMP;
end;
$$
```

Явный курсор

```
declare  
cEMP CURSOR (pSal int) FOR  
SELECT Empno, Ename, Job FROM emp where  
sal >= pSal;  
i record;  
begin  
OPEN cEMP (3000);
```

Явный курсор

-- Первая строка

```
FETCH FIRST FROM cEMP INTO i;  
RAISE NOTICE 'FIRST: Empno =%, Ename =%',  
i.Empno, i.Ename;
```

Явный курсор

-- Следующая строка

```
FETCH NEXT FROM cEMP INTO i;  
RAISE NOTICE 'FIRST: Empno =%, Ename =%',  
i.Empno, i.Ename;
```


Явный курсор

-- Предыдущая строка

```
FETCH PRIOR FROM cEMP INTO i;  
RAISE NOTICE 'FIRST: Empno =%, Ename =%',  
i.Empno, i.Ename;
```

Явный курсор

-- Абсолютная позиция (например 3-я строка)
FETCH **ABSOLUTE** 3 FROM cEMP INTO i;
RAISE NOTICE 'FIRST: Empno =%, Ename =%',
i.Empno, i.Ename;

Явный курсор

-- Относительная позиция (например на 2 строки вперед от текущей)

```
FETCH RELATIVE 2 FROM cEMP INTO i;  RAISE  
NOTICE 'FIRST: Empno =%, Ename =%', i.Empno,  
i.Ename;
```

Явный курсор

-- Последняя строка

```
FETCH LAST FROM cEMP INTO rec;  
RAISE NOTICE 'FIRST: Empno =%, Ename =%',  
i.Empno, i.Ename;
```

Сравнение

Характеристика	Неявный курсор	Явный курсор
Создание	Автоматически при выполнении <code>FOR ... IN ... LOOP</code> ³³	Вручную в блоке <code>DECLARE</code> с помощью <code>CURSOR FOR</code> или <code>refcursor</code> ³³
Управление	Полностью управляется циклом <code>FOR</code> . Не требует команд <code>OPEN</code> , <code>FETCH</code> , <code>CLOSE</code> ³³	Требует явных команд <code>OPEN</code> , <code>FETCH</code> , <code>CLOSE</code> (или <code>MOVE</code>) для управления ³³
Именованение	Не имеет имени, к нему нельзя обратиться напрямую	Имеет имя, которое используется во всех операциях ¹

Сравнение

Гибкость	Ограничен: только последовательный перебор строк в цикле	Высокий контроль: можно перемещаться по строкам (<code>NEXT</code> , <code>PRIOR</code> , <code>ABSOLUTE</code> , <code>RELATIVE</code>), обновлять/удалять текущую строку (<code>WHERE CURRENT OF</code>) ³³
Возврат из функции	Невозможен	Возможен через <code>refcursor</code> для передачи больших наборов данных клиенту ³³
Производительность	Считается предпочтительным и часто более быстрым вариантом ¹¹	Может быть медленнее из-за накладных расходов на ручное управление, но необходим для сложных сценариев [🔗]



Обогащая традиции
инновациями

Спасибо!
У вас остались вопросы?



* изображение для этого слайда создано нейросетью Kandisky 2.2 по запросу «Изображение в серых тонах, которое олицетворяет синергию традиций и инноваций в финансовой сфере»